

Foundations for Quantitative Engineering & Trading

A 90-Day Summer Program — Filtered to What Actually Matters

BEng Mechatronics (UoW Dubai) · Discretionary Trading · Quantitative Finance

0. Before You Start — The Question Nobody Answered

You said something important: you don't know if you'll regret spending these three months on this instead of something else. That's not anxiety — that's rational. It deserves a real answer, not motivation.

Here it is:

The regret question only makes sense if there's a plausible alternative use of your time that you're giving up. If you're asking whether you should be learning trading strategies instead — you're already live trading every day, and this curriculum improves that, not competes with it. If you're asking whether you should be relaxing — that's a real trade-off and only you can weigh it. If you're asking whether this knowledge could become useless — read the Regret Matrix in Section 6 before deciding.

The short answer: every skill in this program transfers to at least three different career paths. The floor on usefulness is very high.

0.5 The 2026 Reality — Anyone Can Get AI to Write Code Now

This needs to be addressed directly, because it changes what's actually worth spending three months on. As of mid-2026: AI coding tools cut the time spent on routine coding tasks by roughly 46% on average. Around 84% of developers now use AI coding assistants daily. Job postings for 'pure implementation' roles — jobs that mainly translate a spec into code — have fallen by about 17% year over year, while postings requiring AI-collaboration skills are up over 300%.

At the same time, the same data shows a problem: code written with heavy AI assistance has measurably more bugs, more duplication, and less refactoring than code written by people who understand what they're doing. The pattern researchers keep finding is that AI removes the cover that used to expose shallow understanding — code that used to take a junior developer all day to produce slowly, with visible struggle, can now be produced instantly and looks fine on the surface, while the person who generated it can't explain why it works or debug it when it breaks.

What this means for this program: the goal was never "learn to type Python." It was always "learn to think in a way that produces correct, justified answers." That goal hasn't changed — if anything it's the only thing that hasn't been commoditized. AI can write the

code. It cannot decide what question is worth asking, verify whether an answer is actually correct, or know enough about finance and statistics to catch a wrong result that looks plausible. Those are exactly the skills this curriculum is built around, and they're becoming more valuable, not less, as code generation gets cheaper.

How This Changes the Way You Should Actually Work

- **Use AI tools as a typing accelerator, never as a substitute for understanding.** It is completely fine — expected, even — to ask Claude or a similar tool to write boilerplate: a CSV loader, a plotting function, API request scaffolding. That is exactly the kind of routine task AI is good at and where resisting it just wastes your time.
- **The rule: if you can't explain every line without looking at it again, you don't own it yet.** Before you commit any AI-generated code, rewrite the core logic from memory in a blank file, or explain it out loud as if teaching it to someone. If you can't, that's a signal to slow down on that piece — not to move on.
- **Do the maths by hand first, then check it in code.** When a week's topic involves a calculation — expected value, a t-statistic, an eigenvalue — work it out on paper before writing or generating any code for it. This is the part AI genuinely cannot do for you, because it's not about output, it's about whether the concept is actually inside your head.
- **Treat every AI-generated answer as a claim to be checked, not a fact to be accepted.** This applies to code and to financial/statistical claims alike. The habit of verifying — checking a result against a second method, sanity-checking against known cases, testing edge cases — is the single most valuable habit in this entire program, and it's explicitly practiced in the research-paper weeks (Month 3), where you check published claims against your own data.
- **This is also why the finance and maths content matters more than ever, not less.** A developer who can only produce code is now competing with a tool that produces code for free. A person who understands finance, probability, and statistics deeply enough to know what to build and whether the output makes sense is not competing with that tool — they're the one operating it.

0.6 Two AIs Disagreed With Each Other — Here's the Actual Decision

One model told you to study Python, then Machine Learning, then Deep Learning, then AI implementation, and only then finance and mathematics. Another told you to optimize for certificates and code every single subsection of a course onto GitHub regardless of relevance. Both are wrong in a specific, explainable way, and you deserve to know why rather than just being handed a third opinion.

The Recommendation	Why It's Wrong For You, Specifically
"Learn ML/Deep Learning/LLMs/AI agents first, then finance."	This inverts the actual prerequisite chain. Machine learning is statistics and linear algebra wearing a costume — you cannot meaningfully understand what a model is doing without probability and linear algebra first, and you haven't done either

The Recommendation	Why It's Wrong For You, Specifically
	yet at the depth required (Stat 110 / MIT 18.06 level). Teaching ML before that produces someone who can call a library function and call it 'doing machine learning' without understanding what it computed or whether to trust it. That is the opposite of the judgment this program is built around (see Section 0.5).
"Optimize for certificates — code every subsection of every course onto GitHub."	This treats course completion as the goal instead of the goal being the goal. Coding every subsection of, say, the Tower of Hanoi or Hash Table sections of a general Python course produces GitHub commits, but commits are not evidence of relevant skill — a hiring manager or your own future self gets no signal from 500 trivial commits that don't connect to finance, trading, or your degree. Volume without relevance is wasted time disguised as productivity.

The actual decision: Math and finance foundations come first, not ML — this hasn't changed from the previous version of this document, and it's reinforced by the prerequisite logic above. Certificates are kept, because you asked for them and they have real value for accountability and for your father, but only for courses that are directly relevant — not for completing every available course regardless of content. Section 0.7 below makes this concrete using your two actual freeCodeCamp options.

0.7 Your Two freeCodeCamp Courses — What to Actually Do With Each

You have two real options sitting in front of you. Here's what's actually inside each one, and the filtered decision for what to do with each.

Option A — "Data Analysis with Python" (28 sections + 5 certification projects)

Contents: Jupyter Notebooks, NumPy (arrays, operations, boolean indexing, algebra), pandas (DataFrames, indexing, conditional selection, creating columns), data cleaning (duplicates, visualization), reading data (CSV, databases, HTML). Five certification projects: Mean-Variance-Standard Deviation Calculator, Demographic Data Analyzer, Medical Data Visualizer, Page View Time Series Visualizer, Sea Level Predictor.

Decision: do all of it. Every section here is directly relevant to financial data work — this is, almost section-for-section, the exact toolkit needed to clean a price history, analyse a trading journal, or build a statistical report. The five certification projects are genuinely good practice for the kind of data manipulation this program needs, even though their titles aren't finance-branded. Complete the full course, claim the certificate, and push every project to GitHub.

Option B — "Python" (531 steps: fundamentals + data structures & algorithms)

Contents: Python Basics, Loops and Sequences, Dictionaries and Sets (+ Build a User Configuration Manager project), Error Handling, Classes and Objects, Object-Oriented Programming (+ Build a Budget App, Build a Polygon Area Calculator), then Linear Data Structures (+ Build a Hash Table), Algorithms (+ Implement the Tower of Hanoi), Graphs and Trees, and Dynamic Programming.

Decision: do the first half, skip the second half — for now. Python Basics through Object-Oriented Programming (including the User Configuration Manager, Budget App, and Polygon Area Calculator projects) builds general programming discipline beyond what MATLAB gave you — proper error handling, classes, and OOP are genuinely useful for writing maintainable code in every project later in this program. Linear Data Structures, Hash Tables, Algorithms, Tower of Hanoi, Graphs and Trees, and Dynamic Programming are CS-interview preparation for software engineering roles — not quant finance roles — and they substantially overlap with CSCI291 (Year 2, Winter), which you will take anyway. Doing them now is duplicated effort against a degree requirement you haven't reached yet, for a skill set that isn't the one this program targets.

What this means for the full certificate: you will not complete the "Python" certification in these 90 days, because finishing it requires the DSA sections you're intentionally skipping. That's the correct trade-off given your goal. If you want that certificate later, the natural time is during or after CSCI291 in Year 2 Winter, when the DSA content reinforces a course you're being graded on instead of competing with it for time.

1. Where You Actually Stand After Year 1

The previous version of this document talked about mechatronics knowledge you don't have yet — signal processing (Year 3), control theory (Year 4), Kalman filters (implied Year 3+). That was wrong. Here's what Year 1 actually gives you, honestly:

Subject Completed	What It Actually Gave You	How It Connects to This Summer
MATH141 — Foundations of Engineering Mathematics	Calculus: derivatives, integrals, limits. Basic linear algebra.	The same calculus is used in finance — the price sensitivity of an option (called delta) is literally a derivative in the calculus sense. You already have the mathematical foundation.
MATH142 — Essentials of Engineering Mathematics	More calculus, differential equations, series.	Differential equations model how prices evolve over time. The most famous finance equation (Black-Scholes) is a differential equation. You've done the prerequisite without realising it.

Subject Completed	What It Actually Gave You	How It Connects to This Summer
PHYS143 — Physics for Engineers	How to build a model of reality with equations and test it against data.	This is the exact method used in quantitative finance. The heat diffusion equation from thermodynamics is mathematically identical to the Black-Scholes option pricing equation. Same maths, different physical system.
ENGG100 — Engineering Computing & Analysis	Programming logic in MATLAB: variables, loops, functions, arrays.	Python uses the same logic. Switching is a syntax change, not a thinking change. You are not starting from zero.
ENGG102/104 — Mechanics & Electrical Systems	Systems thinking. How components interact. How to analyse a system.	Finance is a system. A portfolio is a system. Markets are a system. The habit of thinking in components and interactions transfers directly.
ENGG103 — Materials in Design	Understanding trade-offs under constraints.	Every investment decision is a trade-off under constraints. The mental model is the same.

The key correction from the previous version: The 'Engineering Bridge' section previously suggested connecting signal processing and Kalman filters to finance. Signal processing is ECTE203 — Year 3. Control Theory is ECTE344 — Year 4. You haven't taken these yet. The real bridge is simpler and more honest: your Year 1 calculus and physics directly transfer. That's enough to start, and more will transfer as you progress through Years 2, 3, and 4.

2. Finance, Mathematics, Physics, Philosophy — These Are Not Four Things

You listed four interests as if they're separate. They're not. Here's what they actually are:

Your Interest	What It Fundamentally Is	Where It Appears in Quant Finance
Mathematics	The language of precise reasoning	Finance is written entirely in mathematics. Probability, calculus, linear algebra, statistics — these are not tools applied to finance from outside. They are finance.

Your Interest	What It Fundamentally Is	Where It Appears in Quant Finance
Physics	Building models of reality and testing them against data	Quantitative finance is physics applied to economic behaviour. Several of the most influential quants (Fischer Black, Emmanuel Derman, Jim Simons) came from physics. The methodology is identical: observe → model → test → refine.
Philosophy	Epistemology: how do we know what we know? Decision theory: how should we act under uncertainty?	Every trade is a decision under uncertainty. Bayesian probability — which you will study in Harvard Stat 110 — is literally applied epistemology. The question 'given this evidence, how confident should I be?' is both philosophical and mathematical. Karl Popper's falsifiability applies directly to backtesting.
Finance	Applied mathematics about how value moves through time and under uncertainty	This is the domain. Mathematics, physics, and philosophy are the tools. You already love the tools. You are learning the domain.

The reason you find coding logical and satisfying is the same reason you will find quantitative finance logical and satisfying — it is a domain where things are required to make sense, where claims are tested, and where rigour is rewarded. It is not like most finance, which is opinion-driven. It is like physics: formal, testable, and honest about uncertainty.

3. The 90-Day Program — Overall Roadmap

Days 1–30: The Toolkit	Days 31–60: The Mathematics	Days 61–90: The Application
Python (NumPy, pandas, Matplotlib)	Finance fundamentals — what the domain actually is	APIs: live data into your database
SQL — structured storage for your trade log	Probability — Stat 110 (Harvard)	Research Paper 1: what real return data looks like
Git / GitHub — version control from day one	Statistics & regression applied to your trades	Research Paper 2: technical analysis evaluated academically
freeCodeCamp certification completed	Linear algebra — MIT 18.06	Research Paper 3: trend following formalised

Days 1–30: The Toolkit	Days 31–60: The Mathematics	Days 61–90: The Application
Project 1: Trade Journal Database	Project 2: Statistical Audit of Your Own Trades	Capstone: Quantitative Analysis Engine
		C++ — introduction only, numerics focus

Sequencing logic: Tools first, because without them you cannot do the mathematics in code. Finance fundamentals before statistics, because statistics on returns means nothing if you don't know what a return is. Mathematics before the capstone, because the capstone is the mathematics applied to live data. Nothing appears before its prerequisite is finished.

4. Daily Schedule

Time	Block	What Happens Here
07:30 – 08:30	Live Trading	Execute your discretionary session. Log everything: asset, entry, exit, size, rationale, result. This log is the raw data for Projects 1 and 2.
09:00 – 11:00	Core Curriculum	Watch assigned lectures or read assigned chapters. Take handwritten notes. Do the in-lecture exercises. No coding in this block — only understanding.
11:15 – 13:00	Coding Lab	Translate that day's lecture into code. If the lecture was about probability distributions, write Python that computes and plots one. No new theory — only implementation.
14:00 – 16:00	Project Lab	Build the week's deliverable. This is cumulative — each session adds to the project, not starts a new one. The project is what proves you understood.
16:30 – 17:15	Reading	Month 1: finance reading (Damodaran). Month 2: textbook chapters (Brooks). Month 3: research papers.
17:15 – 17:30	Git Commit + Journal	Commit every file changed today. Write three lines: what you understood today, what confused you, what tomorrow's first task is. This three-line journal is how you track real progress.

Rest rule: One half-day off every week — Sunday afternoon. Every fourth week (end of each month), the Friday Project Lab becomes a full review and refactor session: no new material, only cleaning and improving what already exists. This is where understanding consolidates.

5. Day One — Exactly What You Do

This is the most precise section in the document. Every other section says 'what to learn.' This one says 'what to do at 9:00am tomorrow.' Follow these steps in order.

Setup (do this once, takes about 90 minutes)

1. **Install Python 3.12** from python.org — not from the Microsoft Store, not via Anaconda yet. Tick the box 'Add Python to PATH' during installation.
2. **Install VS Code** from code.visualstudio.com. Open it. Install these two extensions from the Extensions panel: 'Python' (by Microsoft) and 'Jupyter' (by Microsoft).
3. **Create a GitHub account** at github.com if you don't have one. Use a professional username — this becomes your public portfolio.
4. **Create your first repository.** Name it exactly: quant-foundations. Set it to Public. Tick 'Add a README file.' Click Create.
5. **Install Git** from git-scm.com. During installation, accept all defaults.
6. **Open a terminal in VS Code** (View → Terminal). Run these commands one at a time:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

```
git clone https://github.com/YOURUSERNAME/quant-foundations
```

```
cd quant-foundations
```

7. **Install the Python packages you will use this month:**

```
pip install numpy pandas matplotlib seaborn jupyter
```

First Learning Block (09:00 – 11:00)

8. **Go to freecodecamp.org/learn.** Navigate to: Certifications → Data Analysis with Python. Click 'Start.'
9. **Watch the first 90 minutes of video content.** Take notes on paper, not on screen. Write down anything you don't understand — you will search for it in the coding block.

First Coding Block (11:15 – 13:00)

10. **Create a file:** quant-foundations/week1/day1.py
11. **In that file, translate three things from the lecture into working code.** For example, if the lecture covered NumPy arrays, write a script that: creates an array of 30 days of simulated prices, computes the daily percentage change, prints the mean and standard deviation.

12. **If you get stuck on syntax, it's fine to ask an AI assistant (Claude, ChatGPT, Copilot) to write the line for you.** That's a reasonable use of the tool. But before moving to the next step, close the AI response and rewrite that exact line from memory in a comment above it. If you can't, re-read the lecture section on that concept — don't just keep pasting.

13. **Run it. Fix errors. Run it again until it works.**

14. **At 17:15, open terminal and run:**

`git add .`

`git commit -m "Day 1: first NumPy script"`

`git push`

How you know Day 1 was successful: You have a GitHub repository with one committed Python file. That file runs without errors. You wrote three lines in your journal. This is the minimum. If you also understood the lecture content well enough to explain it in one sentence, that is ideal. If not, that's what tomorrow morning's 09:00 block is for.

6. The Regret Matrix — An Honest Answer to 'Will I Waste My Time'

For each skill in this program, here is what happens in four different futures:

Skill	If You Become a Quant / Trader	If You Stay in Engineering	If You Go Somewhere Completely Else	Regret Risk
Python + pandas	Daily core tool	Used in modern engineering everywhere: data acquisition, simulation, automation	One of the most employable technical skills in the world	Very low
SQL	Important for data pipelines	Engineering databases, test data storage	Used in almost every data-handling role	Very low
Statistics & probability	Core — you cannot function without it	STAT291 (Year 3) becomes significantly easier	Universally applicable to any data-driven role	Essentially zero
Finance fundamentals	Core domain knowledge	Personal financial literacy, investment decisions	Personal financial literacy — this improves your	Zero

Skill	If You Become a Quant / Trader	If You Stay in Engineering	If You Go Somewhere Completely Else	Regret Risk
			own life regardless	
Linear algebra	Core — portfolios, factor models, everything	Already embedded in ECTE203 (Year 3) and ECTE344 (Year 4) — you'll relearn it anyway	Less applicable, but not wasted	Low
Git / GitHub	Standard — every professional uses this	Standard in every engineering software role	Standard in every technical role	Zero — this is just how technical work is shared
Research paper reading	Essential for staying current	Essential for thesis and research projects	Builds critical thinking applicable anywhere	Zero
Judgment: verifying AI output, knowing what's correct	The single most valuable skill in 2026 — see Section 0.5	Directly transfers to checking simulation/analysis results in any engineering role	Transfers to literally any field involving AI tools, which by 2026 is most of them	Essentially zero — this is the one skill getting more valuable, not less, as AI improves

The honest conclusion: There is no skill in this program that you will regret learning. The only real question is whether you'd prefer to spend July, August, and half of September doing something else entirely. If the answer is no — if you'd be sitting at home anyway — then this is the highest-leverage thing you can do with that time. If you genuinely need rest, take August as a lighter month.

7. Week-by-Week Curriculum

Month 1 — The Toolkit (Days 1–30)

Goal by Day 30: You have a working SQL database storing your real trades. You have a Python pipeline that cleans and loads data. You have 30 days of GitHub commit history. freeCodeCamp Data Analysis with Python certification is complete, and the relevant (filtered) portion of the Python fundamentals course is complete with three small portfolio projects.

Week 1

Topic: Python foundations beyond MATLAB — proper error handling, classes, and object-oriented programming

Why it belongs here: *MATLAB gave you variables, loops, and functions. It did not give you OOP discipline or error handling — both of which make every later project (the database pipeline, the API downloader, the capstone) easier to write correctly and easier to debug.*

Resource: freeCodeCamp — "Python" course, filtered to: Python Basics → Loops and Sequences → Dictionaries and Sets → Error Handling → Classes and Objects → Object-Oriented Programming. Skip everything after OOP (see Section 0.7 for why).

Exercise: Complete the three certification projects inside this filtered range: Build a User Configuration Manager, Build a Budget App, Build a Polygon Area Calculator.

Project work: All three projects refactored slightly beyond the course's minimum requirement — add proper error handling and at least one class to each, even where the course doesn't strictly require it.

Deliverable: Three project folders committed to GitHub, each with a short README explaining what it does and one paragraph on what error handling or OOP choice you made and why.

Week 2

Topic: Data analysis tooling — Jupyter, NumPy, pandas, data cleaning, visualisation

Why it belongs here: *This is the actual toolkit for financial data work. Every section in this course maps directly onto cleaning a price history or analysing a trading journal, even though the course itself isn't finance-branded.*

Resource: freeCodeCamp — "Data Analysis with Python" Certification, full course (100% free, certificate included, no paywall at any stage)

Exercise: Work through all 28 core sections in order: Jupyter Notebooks → NumPy (arrays, operations, boolean indexing, algebra) → pandas (DataFrames, indexing, conditional selection, creating columns) → data cleaning → reading data (CSV, databases, HTML).

Project work: Complete all five certification projects: Mean-Variance-Standard Deviation Calculator, Demographic Data Analyzer, Medical Data Visualizer, Page View Time Series Visualizer, Sea Level Predictor.

Deliverable: freeCodeCamp Data Analysis with Python certificate claimed. All five projects committed to GitHub in their own folders, each with a README.

Week 3

Topic: SQL — relational schema design, DDL, DML, queries

Why it belongs here: *Your trade log needs structure before statistics on it mean anything. A spreadsheet has no integrity constraints. A database does.*

Resource: CS50's Introduction to Databases with SQL — Harvard (free to audit and complete all problem sets; free non-verified certificate at 70%+; optional one-time \$299 for verified Harvard credential)

Exercise: Design a 3-table schema on paper first: trades (trade_id, date, asset, direction, entry, exit, result), assets (asset_id, name, type), strategies (strategy_id, name, description). Then write the SQL CREATE statements.

Project work: Schema for the Trading Journal database: initialised .db file with all tables created and verified.

Deliverable: ERD diagram image saved to repo. Initialised .db file committed. CS50 SQL: first 3 problem sets submitted at 70%+.

Certificate note: *Free CS50 Certificate requires all 9 problem sets at 70%+. Complete the remaining 6 in Week 4. Verified Harvard certificate (\$299 one-time) is the one worth considering if a visible credential matters.*

Week 4

Topic: Python ↔ SQL integration — sqlite3, parameterised queries, idempotent ingestion

Why it belongs here: *This is the pipe that turns your daily 7:30am trade log into permanently queryable data. It runs once and you never manually enter a trade again.*

Resource: Python official sqlite3 documentation (free) — once SQL is solid from CS50, this is a lookup, not a course

Exercise: Write a script that reads a CSV of trades, connects to the database, and inserts only rows that don't already exist (idempotent — safe to rerun).

Project work: Project 1 (Major): Trading Journal Ledger Database — complete ingestion pipeline, your historical trades loaded, queries running.

Deliverable: Working Python ingestion script. All historical trades in the database. Two SQL queries running: total P&L by asset, win rate by month.

Certificate note: *Complete remaining CS50 SQL problem sets this week to earn the free certificate.*

Month 2 — The Mathematics Applied (Days 31–60)

Goal by Day 60: You can run a formal statistical audit of your own trading performance — p-values, Sharpe ratio, Sortino ratio, max drawdown, OLS regression — automatically from your database. You understand what each number means and why it matters.

Week 5

Topic: Finance fundamentals — what financial instruments actually are, how markets work, what financial statements mean

Why it belongs here: *This is the layer all three original AI drafts skipped despite finance knowledge starting at approximately 1%. Statistical analysis of returns means nothing if you don't know what a return is, what a bond is, or why a derivative exists.*

Resource: Aswath Damodaran's Corporate Finance lectures — NYU Stern (free on YouTube and his personal site damodaran.com — no account, no subscription, no certificate, no cost)

Exercise: Read the annual report of one company you find interesting. Identify revenue, operating profit, net income, total assets, total equity. Write a one-paragraph plain-English description of what the company does financially.

Project work: Finance glossary: a document in your repo mapping 25 finance terms to plain English with one concrete example each. This becomes a reference you use for the rest of the program.

Deliverable: Markdown glossary committed to repo. Damodaran: first four lecture topics covered.

Week 6

Topic: Probability — random variables, distributions, expectation, variance, Bayes' theorem

Why it belongs here: *Every statement about trading performance is a probability statement. Your win rate is a probability. Your expected value is an expectation. Your risk is a variance. You cannot analyse your trading rigorously without this.*

Resource: Harvard Stat 110 — Probability (Blitzstein) — fully free on the course website and YouTube; no login, no account, no subscription, no paywall at any stage

Exercise: Compute the expected value and variance of your trade P&L distribution using your database data. Plot it in Python. Compare its shape to a normal distribution.

Project work: Probability profile of your trading outcomes: distribution plot, mean, variance, skewness, kurtosis. Stored as a Jupyter notebook in the repo.

Deliverable: Notebook with distribution plots committed. Stat 110: probability fundamentals and distributions lectures complete.

Week 7

Topic: Statistical inference — hypothesis testing, confidence intervals, t-tests, regression introduction

Why it belongs here: *The most important question in trading is: is this edge real, or is it luck? That question has a mathematical answer. A hypothesis test gives it to you.*

Resource: Introductory Econometrics for Finance — Brooks (book, one-time purchase or library/PDF copy), Chapters 2–3

Exercise: State the null hypothesis: 'My true win rate has zero positive expectancy.' Compute the t-statistic. Find the p-value. Interpret it in plain English.

Project work: Trading Strategy Significance Evaluator: a Python script that pulls your trade data from the database and runs a formal t-test, returning the t-statistic, p-value, and a plain-English interpretation.

Deliverable: Script committed and running on real trade data. Brooks Chapters 2–3 read and summarised.

Week 8

Topic: Linear algebra — vectors, matrices, covariance, eigenvalues, OLS regression. Git workflow formalised.

Why it belongs here: *Portfolio risk is a covariance matrix. OLS regression is matrix algebra. These are not advanced topics — your MATH141/142 already gave you the foundation. This applies it to finance.*

Resource: MIT OpenCourseWare 18.06 Linear Algebra — Gilbert Strang (completely free, no login, no account, downloadable lectures, problem sets with solutions)

Exercise: Compute the covariance matrix of the daily returns of three assets you trade. Find its eigenvalues. Interpret what the largest eigenvalue represents.

Project work: Project 2 (Major): Discretionary Performance Econometric Auditor — rolling Sharpe ratio, Sortino ratio, maximum drawdown, OLS alpha versus a benchmark index, all pulled automatically from the Month 1 database.

Deliverable: Public GitHub repo with documented code and automated output summary. MIT 18.06: first 10 lectures and corresponding problem sets complete.

Certificate note: *No certificate from MIT OCW. The project repository is the proof of mastery. The skills transfer to STAT291 (Year 3) and ECTE344 Control Theory (Year 4).*

Month 3 — The Application (Days 61–90)

Goal by Day 90: A live data pipeline running. Three research papers critically read and documented. A capstone analysis engine that combines everything built in Months 1 and 2.

Week 9

Topic: APIs — REST architecture, JSON parsing, authentication headers, rate limiting, automated data collection

Why it belongs here: *Manual CSV downloads don't scale and aren't repeatable. A real data pipeline fetches live data automatically and stores it directly into your database.*

Resource: The data provider's own developer documentation (Yahoo Finance API, EODHD, or Binance API — all free tiers available, no subscription required for historical data)

Exercise: Build a Python wrapper that downloads 1-minute candle data for one asset, handles rate limits gracefully, and writes the result directly into the database from Month 1.

Project work: Multi-asset data downloader: a modular script that takes a list of tickers and a date range as inputs and delivers clean data to the database.

Deliverable: Script committed and running successfully on at least three real assets. Database now contains both manual historical data and auto-fetched live data.

Week 10

Topic: Research Paper 1 — the actual statistical properties of real return data

Why it belongs here: *Before trusting any analysis you run on price data, you need to know what real return data actually looks like. This paper defines the benchmark. Many of your Month 2 statistics assumed normally distributed returns — this paper tells you how wrong that assumption is and why it matters.*

Resource: Cont, R. (2001) — 'Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues' (free via SSRN or Google Scholar)

Exercise: Identify each 'stylized fact' the paper describes — fat tails, volatility clustering, absence of autocorrelation in returns. Check whether each one appears in your own traded assets' data.

Project work: Paper analysis report: a markdown document structured as: claim from paper → test on your data → result → implication for your earlier statistics from Month 2.

Deliverable: Markdown report committed. At least 4 stylized facts tested on real data.

Week 11

Topic: Research Paper 2 — academic evaluation of technical analysis. Capstone begins.

Why it belongs here: *This paper directly examines whether the tools discretionary traders use have empirical support. Reading it as a trader — not as a believer or a sceptic, but as someone who can now read a statistical methodology — changes how you see your own decisions.*

Resource: Park & Irwin (2007) — 'The Profitability of Technical Analysis: A Review' (free via SSRN or Google Scholar)

Exercise: Identify the two main biases the paper identifies in technical analysis research: data-snooping bias and survivorship bias. Write a one-paragraph explanation of each in your own words.

Project work: Begin capstone: Quantitative Analysis Engine. This session: set up the repository structure, connect the API downloader to the database, and run the Month 2 statistical audit on live fetched data (not historical CSV).

Deliverable: Paper 2 methodology breakdown document committed. Capstone repo initialised with working data flow from API to database to statistical output.

Week 12

Topic: Research Paper 3 — trend following formalised. Capstone complete.

Why it belongs here: *This paper provides a mathematical formalisation of trend following across a century of data. It is the paper you will partially replicate in the capstone, and the one that connects your work as a discretionary trader to quantitative methodology.*

Resource: Hurst, Ooi & Pedersen (2013) — 'A Century of Evidence on Trend Following Investing' (free via AQR's website or SSRN)

Exercise: Extract the paper's core trend signal calculation. Implement it in Python on one asset class. Compare your implementation's output to what the paper reports.

Project work: Capstone complete: Quantitative Analysis Engine — live data ingestion via API, storage in SQL database, statistical audit of your own trades, trend signal implementation from Paper 3, publication-style comparison report in Markdown.

Deliverable: Full public GitHub repository: structured code, documentation, test on real data, final report. This is the centrepiece of your portfolio going into Year 2 internship applications.

8. Required Resources

One resource per topic. No subscriptions. Either 100% free or free to learn with a one-time optional certificate fee.

Topic	Resource	Cost	What You Actually Get
Python fundamentals (OOP, error handling)	freeCodeCamp — "Python" course, filtered to Basics → OOP only (see Section 0.7)	100% free for the portion used; full certificate not pursued (requires DSA sections, deferred to Year 2)	Three small certification-style projects (Config Manager, Budget App, Polygon Calculator) plus the coding discipline MATLAB never taught.
NumPy / pandas / data analysis	freeCodeCamp — Data Analysis with Python Certification	100% free including the certificate	5 real data-analysis projects as deliverables. Certificate with your name on it, permanent URL. No audit-vs-pay split.
SQL	CS50's Introduction to Databases with SQL — Harvard (via edX or Harvard Online)	Free to learn. Free non-verified certificate at 70%+. Optional verified certificate: one-time \$299.	Harvard problem sets, real schema design, real query writing. The one certificate worth potentially paying for — a single payment, not recurring.
Git / GitHub	Pro Git (official free book, git-scm.com/book) + GitHub Skills (free interactive course at skills.github.com)	100% free	The canonical reference plus hands-on practice in real GitHub repositories.
Finance fundamentals	Aswath Damodaran — Corporate Finance lectures (damodaran.com and YouTube)	100% free	The actual reference material used by finance professionals. More rigorous than any paid 'intro to finance' course.
Probability	Harvard Stat 110 — Blitzstein (stat110.net and YouTube)	100% free	The most respected free probability course available. No login required at any point.
Statistics & regression for finance	Introductory Econometrics for Finance — Brooks (book)	One-time book purchase. PDF or	The standard academic text for applying statistics specifically to financial return data. Not a

Topic	Resource	Cost	What You Actually Get
		library copy acceptable.	general stats textbook — directly relevant.
Linear algebra	MIT OpenCourseWare 18.06 — Gilbert Strang (ocw.mit.edu)	100% free	The actual MIT course. Lectures, problem sets, and solutions all downloadable. No account required.
C++ (light, Month 3 only)	QuantStart — free C++ for quant finance articles (quantstart.com)	100% free	Specifically aimed at quantitative development use cases, not general-purpose programming.
APIs / live data	Provider's own developer documentation	100% free	This is a build-it skill, not a course skill. Read the docs, build the thing.

9. Books

Introductory Econometrics for Finance — Chris Brooks (required, one-time purchase). This is the book that bridges statistics and financial data specifically. It does not require prior econometrics knowledge, and it does not cover abstract statistical theory for its own sake. Every chapter is framed around financial applications. It is used in the Week 7 and Week 8 curriculum.

Python for Data Analysis — Wes McKinney (optional, one-time purchase or library copy). freeCodeCamp already covers what Weeks 1–2 need. Buy this if you want a deeper technical reference once you're further along. It is the most authoritative reference because McKinney created pandas.

No other books are added. The original drafts listed more; most of them would compete with project time without adding proportional value at this stage.

10. Research Papers — Reading Order and Why

These are read in the order shown because each one builds on the mathematical literacy developed in the preceding weeks.

Order	Paper	Read in	Why This Order	What to Extract
1	Cont, R. (2001). 'Empirical Properties of Asset Returns.'	Week 10	Requires only the Week 6–7 statistics. Establishes what real data actually looks like before you trust your own analysis.	The list of stylized facts. Test each one on your own traded assets.
2	Park & Irwin (2007). 'The Profitability of Technical Analysis: A Review.'	Week 11	Low mathematical bar. Directly relevant to discretionary trading. Teaches how academic research evaluates trading claims.	Data-snooping bias and survivorship bias — what they are, how they distort results, how to control for them.
3	Hurst, Ooi & Pedersen (2013). 'A Century of Evidence on Trend Following Investing.'	Week 12	Assumes return-distribution literacy from Papers 1–2 and the statistics from Month 2. The capstone replicates part of this paper.	The exact mathematical trend signal formulation. The multi-asset implementation. The performance metrics used.

For each paper, write down three things: (a) what data they used and where it came from, (b) what one assumption, if false, would completely invalidate the result, (c) one thing you can test on your own trade data using the same method.

11. Projects — GitHub Portfolio

Five projects, increasing in complexity. All live in one monorepo (quant-foundations/) with a folder per project, each with its own README.

Project	Weeks	What It Is	Why It Matters
1 — Trading Journal Ledger Database	3–4	A normalised SQL database (trades, assets, strategies tables) with a Python ingestion pipeline that loads your real historical trade log idempotently.	This is the data foundation everything else runs on. Every subsequent project queries this database. It also immediately improves your daily trading practice — structured logging forces clearer thinking about each trade.

Project	Weeks	What It Is	Why It Matters
2 — Trading Strategy Significance Evaluator	7	A Python script that pulls your trade data from Project 1 and runs a formal hypothesis test: is the win rate statistically distinguishable from random? Returns t-statistic, p-value, confidence interval, plain-English interpretation.	This is the first time you will know, with mathematical rigour, whether your edge is real or luck. The result may be uncomfortable. It is more valuable if it is uncomfortable.
3 — Discretionary Performance Econometric Auditor	8	A full statistical performance report generated automatically from your database: rolling Sharpe ratio, Sortino ratio, maximum drawdown, OLS alpha versus a benchmark. Runs with a single command.	This is the kind of performance analysis done at hedge funds. Running it on your own trades gives you the same diagnostic capability applied to your own performance.
4 — Multi-Asset Data Downloader	9	A Python script that fetches live historical OHLCV data via API for multiple tickers, handles rate limits, and writes directly to the Project 1 database. Repeatable and modular.	Removes the bottleneck of manual data management. Every future project can run on fresh live data automatically.
5 — Capstone: Quantitative Analysis Engine	11–12	A complete system: live data via API → database storage → statistical audit (from Project 3) → trend signal implementation (from Paper 3) → formatted comparison report. Structured as a professional software repository with documentation.	This is the centrepiece of your GitHub portfolio. It demonstrates the ability to combine data engineering, statistics, and research replication in a single coherent system — the core skill expected in quant research and quant dev roles.

12. How This Summer Feeds Into Your Actual Degree

This is a new section not in any previous version. Every subject you're taking in Year 2 has a connection to what you're doing this summer. This is not a coincidence — it was designed this way.

Year 2 Subject	When	How This Summer Prepares You
CSCI291 — Programming for Engineers	Winter	Python and MATLAB share the same programming logic: variables, loops, functions, arrays, file I/O. C++ (which CSCI291 likely uses) follows the same logic

Year 2 Subject	When	How This Summer Prepares You
		again. Three months of daily Python coding means the logic is automatic — only the syntax is new. Students who haven't programmed all summer will be rebuilding the habit.
MATH291 — Advanced Engineering Mathematics	Spring	This course likely covers Fourier transforms, Laplace transforms, ODEs, PDEs, and series. The mathematical maturity you build studying probability (Stat 110) and linear algebra (MIT 18.06) makes these topics significantly easier to absorb. You'll also have coded mathematical operations for three months — numerical intuition.
STAT291 — Engineering Statistics	Year 3 (Winter)	This is a direct overlap. The hypothesis testing, confidence intervals, distributions, and regression you cover in Weeks 6–7 of this program are the same topics STAT291 covers. You will have already done this with real data and built a working implementation of it. STAT291 will consolidate rather than introduce.
ECTE203 — Digital Signal Processing	Year 3 (Autumn)	Not directly covered this summer — and that was a correction made to this version. However: the mathematical maturity from Stat 110 and MIT 18.06, combined with the pattern of reading technical material carefully (from the research papers), means you enter ECTE203 better prepared to handle the mathematics of signals.
ECTE344 — Control Theory	Year 4 (Winter)	Control theory is linear algebra applied to dynamical systems. The eigenvalue decomposition you compute in Week 8 (for portfolio covariance) is the same mathematical operation used to analyse system stability in control theory. The connection is real, not metaphorical.
ECTE498 — Thesis A	Year 4	Every skill built this summer — research paper reading, data collection via APIs, statistical analysis, Python implementation, version control, formal technical writing — is directly applicable to thesis work. Students who learn these skills in Year 4 when the thesis starts are learning them too late.

The point: This summer is not a detour from your degree. It is a parallel track that reinforces it. The only subject in Year 2 with no significant overlap is ENGG251 (Mechanics of Solids) and MECH226 (Machine Dynamics) — these are purely mechanical and are not the focus of your long-term goals anyway.

13. Milestones

Day 30 — End of July

- **freeCodeCamp Data Analysis with Python certification:** complete or within one week of completion.
- **GitHub repository:** 30+ commits, readable history, Project 1 in its own folder with documentation.
- **Trading Journal Database:** running locally, all historical trades loaded, two meaningful queries producing correct output.
- **CS50 SQL:** all 9 problem sets submitted. Free CS50 certificate available.
- You can write a Python script that loads data, cleans it, and stores it in a database without looking anything up.

Day 60 — End of August

- **Finance literacy:** you can explain what a stock, bond, derivative, balance sheet, income statement, and P&L are in plain language without hesitation.
- **Trading performance audit:** Project 3 is running on your real trade data. You know your actual Sharpe ratio, Sortino ratio, and maximum drawdown. You know whether your edge is statistically significant.
- **Mathematics:** Harvard Stat 110 fundamentals complete. MIT 18.06 first 10 lectures complete. You can compute a covariance matrix and interpret its eigenvalues.
- **GitHub portfolio:** 60+ commits. Projects 1, 2, and 3 all in the repo with READMEs.

Day 90 — Mid-September

- **Live data pipeline:** API downloader running, data flowing into database automatically.
- **Three papers read:** each has a written methodology breakdown committed to the repo.
- **Capstone complete:** Quantitative Analysis Engine is public, documented, and produces a real output on live data. This is your strongest single artefact for any future internship or research application.
- **You are ready for Year 2:** CSCI291 (Programming for Engineers) will be significantly easier. MATH291 (Advanced Engineering Mathematics) will be easier. STAT291 (Year 3) you will have effectively already done.

14. Execution Rules

- **No new material starts until the current week's deliverable is shipped.** If Week 2's project is incomplete on Monday of Week 3, Monday is for finishing Week 2, not starting Week 3.
- **Every script gets committed the day it's written.** A script that exists only on your laptop does not count. A commit is the record.

- **The three-line journal happens every day.** Not a long reflection — three lines: what you understood, what confused you, what tomorrow starts with. This is how you track real progress versus the feeling of progress.
- **Sunday afternoon is always off.** Not optional.
- **The fourth week of every month is review and refactor.** No new topics. Only improving what already exists.
- **If you miss a day, the next day starts earlier or ends later.** The schedule doesn't slide — the work gets absorbed. One missed day doesn't derail a week.
- **AI tools are allowed and encouraged for boilerplate, never as a substitute for the explain-it-from-memory test in Section 0.5.** Every project deliverable should be something you can walk through line by line, out loud, without checking notes — regardless of how much AI assistance helped you write it the first time.

A final note on logic and sense: You said you like things that make logical sense. Quantitative finance at this level is entirely about making financial phenomena make logical sense — removing the noise, testing claims rigorously, and understanding exactly why something works or doesn't. This is not the finance of TV commentary or social media traders. It is closer to physics: formal, testable, and honest about what it doesn't know. That is exactly the kind of thing you described liking. And in a 2026 job market where AI can generate code on demand, that kind of rigorous, verifiable thinking — not the typing — is the part nobody can automate away.